



Práctica: *Programación Lineal*.
Heurística y Optimización
Curso 2026-2027

1. Objetivo

El objetivo de esta práctica es que el alumno aprenda a *modelar* y *resolver* tareas de Programación Lineal desarrollando programas que integren librerías de modelización y resolución.

Importante: la práctica **debe** hacerse en grupos de dos estudiantes. Si no has encontrado alguien con quien hacerla, ponlo en conocimiento de tu profesor inmediatamente.

2. Enunciado del problema

A continuación se solicita resolver diferentes problemas de optimización cuya solución debe darse, para cada parte, con dos ficheros:

- Fichero de modelo (`.mod`): contiene el modelo *general* que resuelve la tarea de optimización, **para cualquier caso**.

El nombre de este fichero se especifica en la descripción de cada parte a continuación.

- *Script* desarrollado en Python 3.12 que resuelva óptimamente el problema indicado en un fichero de entrada, genere información sobre la solución encontrada en la salida estándar, y muestre la solución óptima en un fichero de salida.

El nombre del *script* se especifica en la descripción de cada parte a continuación.

El *script* debe recibir exactamente dos argumentos en el siguiente orden: `fichero-entrada`, y `fichero-salida`. A continuación se muestran diferentes casos de uso:

```
$ ./sol-1.py ejemplo.in ejemplo.dat
$ ./sol-1.py /tmp/ejemplo.in ejemplo.dat
$ ./sol-1.py ejemplo.in ../../datos/ejemplo.dat
$ ./sol-1.py /tmp/ejemplo.in ../../datos/ejemplo.dat
```

Como se ve, cada fichero puede indicarse con una ruta absoluta o relativa, y si no contiene ninguna entonces se asume aquella en la que se encuentra el *script*, que debe:

- Generar un fichero de datos `.dat`, con el nombre y la extensión indicada como segundo argumento, que codifique la información dada en el fichero de entrada, de acuerdo a las decisiones de diseño tomadas en el modelo implementado en el fichero `.mod`. Este fichero no debe ser borrado por el *script*.
- Resolver óptimamente la tarea correspondiente al caso de entrada, invocando el *solver* de GLPK con el modelo y el fichero de datos generado.
- Mostrar en pantalla la siguiente información de la solución obtenida en una única línea: número de variables creadas; número de restricciones creadas; y el valor de la función objetivo. Por ejemplo:

```
$ ./sol-1.py ejemplo.in ejemplo.dat
17 83 19.33
```

- Generar un fichero de salida que muestre gráficamente la solución óptima encontrada en formato libre pero comprensible.

Se anima a todos los estudiantes a intentar resolver la práctica usando cualquier otro lenguaje de programación (C++, Go, Rust, ...). En ese caso:

- Pregunta a tu profesor sobre el lenguaje elegido, para asegurar que los ordenadores que se usen para evaluar tus soluciones disponen del *software* necesario para la compilación o interpretación de tu código.
- De ninguna manera debe distribuirse el ejecutable que implementa la solución.
- La solución de cada parte debe acompañarse de un fichero `Makefile` que, en ningún caso debe instalar software ni solicitar permisos de superusuario. Su único propósito debe ser el de generar, en el mismo directorio donde es invocado, el ejecutable que resuelve la parte correspondiente de la práctica.
- El script de Python simplemente deberá invocar el ejecutable que resulte de la compilación del código con los parámetros correctos, de modo que el efecto de invocar el *script* sea el indicado más arriba.

2.1. Parte 1: Minimización del coste medio

Una compañía de logística debe disponer cajas todos los días sobre *pallets* cuadrangulares de lado l , y desea determinar la disposición de las cajas de acuerdo a diferentes restricciones y usos.

En la primera parte, se solicita determinar la disposición óptima de n cajas del mismo tamaño y peso sobre un *pallet* cuadrangular de lado l . Así mismo, las cajas son también cuadrangulares con un lado igual a la unidad, de modo que sobre el *pallet* caben hasta l^2 cajas. Las cajas se distinguen únicamente por una prioridad diferente, p_i , que es un número entero, donde 1 es la prioridad máxima, y, en general la prioridad de la caja i -ésima, p_i , es mayor que la de la caja j -ésima si y sólo si $p_i < p_j$.

Las cajas se deben disponer en un único plano sobre el *pallet* en el que sólo hay un punto de acceso como se muestra en la vista cenital de la Figura 1. Además, como también se muestra en esta figura, *cada caja debe ubicarse por delante, respecto al punto de acceso, de aquellas que tienen menos prioridad que ella*.

El coste de la disposición de una caja en la fila i -ésima, y la columna j -ésima, (i, j) se define como la suma de las prioridades de todas las cajas en la misma columna que hay antes de ella. Denominando $(1, 1)$ a la esquina inferior izquierda, de modo que el primer índice represente la fila, y el segundo la columna, entonces el coste de la disposición de una caja en la posición (i, j) es:

$$c_{i,j} = \sum_{i'=i+1}^l p_{i',j}$$

donde $p_{i,j}$ es la prioridad de la caja dispuesta en la posición (i, j) .

Para la resolución de esta parte, se recibirá un fichero con el formato:

```
<longitud del lado del pallet, l>
<p1 p2 ... pn>
```

donde la segunda fila especifica la prioridad de cada una de las n cajas, con $n = l^2$. Esto es, el *pallet* se rellena siempre completamente. Por ejemplo, los contenidos del fichero de entrada cuya solución se muestra en la Figura 1 podrían ser:

```
3
4 11 13 8 5 4 1 3 9
```

	Col 1	Col 2	Col 3
Fila 3	1	3	4
Fila 2	4	8	5
Fila 1	11	9	13

Figura 1: Disposición correcta de 9 cajas sobre un *pallet* de lado $l = 3$ con un único acceso. Cada caja muestra su prioridad en el centro.

Se pide:

1. Modelar la tarea de optimización con un modelo general, con Programación Lineal entera, que minimice el coste medio de la disposición de todas las cajas, indicando claramente el tipo de problema de qué se trata, el significado de cada variable de decisión, el propósito de cada restricción, y la formalización de la función objetivo.

Está expresamente prohibida la escritura de restricciones, o la función objetivo con operadores que no sean puramente lineales.

2. Implementar un modelo general que sirva para resolver la tarea de optimización propuesta, que se debe llamar `parte-1.mod`.
3. Desarrollar un *script* en Python llamado `sol-1.py`, de acuerdo a las indicaciones dadas, que sirva para generar ficheros de datos `.dat`, y para resolver óptimamente la tarea de Programación Lineal resultante.

2.2. Parte 2: Montaje volumétrico

En este caso se deben considerar las mismas restricciones y función objetivo que en el apartado anterior, pero esta vez se pueden montar una cantidad arbitraria h de niveles de cajas sobre el *pallet*. Las restricciones de prioridad aplican únicamente a las cajas en el mismo nivel, de modo que las prioridades de cajas entre diferentes niveles no se afectan mutuamente de ninguna manera.

En esta parte, todas las cajas tienen el mismo tamaño, pero se distinguen por un peso ω_i , y una capacidad de carga γ_i , $1 \leq i \leq n$, de modo que debe garantizarse que cada una soporta un peso, definido como la suma del peso de todas las cajas en la misma fila y columna sobre ella, que debe ser menor o igual que su propia capacidad.

El coste de la disposición de una caja no depende del nivel en el que se coloque, y se aplica la misma expresión de la parte anterior, donde sólo se tienen en cuenta la fila y columna.

Para la resolución de esta parte, se recibirá un fichero con el formato:

```
<longitud del lado del pallet, l> <número de niveles, h>
<p1 p2 ... pn >
<ω1 ω2 ... ωn >
<γ1 γ2 ... γn >
```

donde la segunda fila especifica la prioridad de cada una de las n cajas, con $n = hl^2$, y la tercera y cuarta fila indican su peso, ω , y capacidad, γ en gramos. Esto es, el *pallet* se rellena siempre completamente con la cantidad dada de niveles.

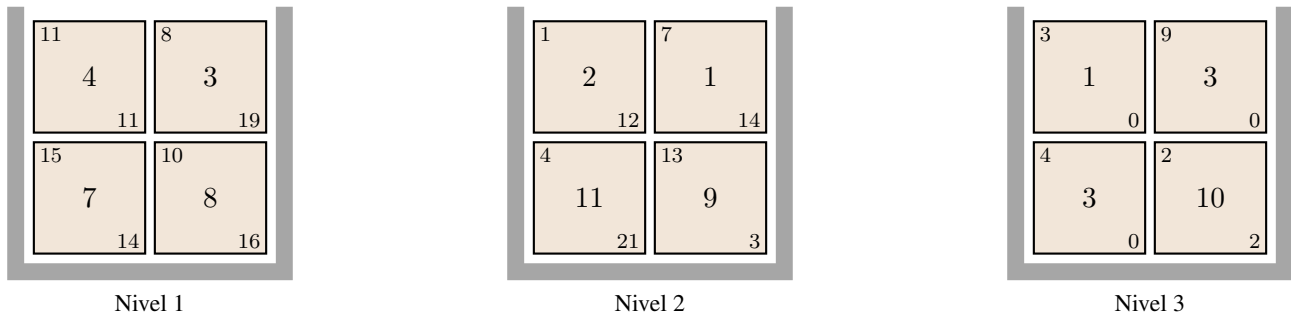


Figura 2: Disposición óptima de 12 cajas sobre un *pallet* de lado $l = 2$ con tres niveles y un único lado de acceso. Cada caja muestra su prioridad en el centro, su peso en la esquina superior izquierda y su capacidad en la esquina inferior derecha, ambos en gramos. El nivel 1 es el inferior y el nivel 3 es el superior.

Por ejemplo, con los datos de entrada:

```
2 3
1 3 3 4 9 11 7 10 3 2 1 8
3 4 9 11 13 4 15 2 8 1 7 10
0 0 0 11 3 21 14 2 19 12 14 16
```

la solución óptima es la que se muestra en la Figura 2.

Se pide:

1. Modelar la tarea de optimización con un modelo general, con Programación Lineal entera, que minimice el coste medio de la disposición de todas las cajas, indicando claramente el tipo de problema de qué se trata, el significado de cada variable de decisión, el propósito de cada restricción, y la formalización de la función objetivo.
Está expresamente prohibida la escritura de restricciones, o la función objetivo con operadores que no sean puramente lineales.
2. Implementar un modelo general que sirva para resolver la tarea de optimización propuesta, que se debe llamar `parte-2.mod`.
3. Desarrollar un *script* en Python llamado `sol-2.py`, de acuerdo a las indicaciones dadas, que sirva para generar ficheros de datos `.dat`, y para resolver óptimamente la tarea de Programación Lineal resultante.

2.3. Parte 3: Optimización de *pallets* incompletos

En la tercera parte, debe considerarse exactamente el mismo caso de la parte anterior, pero esta vez el número de cajas, n , puede ser menor que hl^2 , $1 \leq n \leq hl^2$, donde h es el número máximo de niveles que pueden disponerse. Por lo tanto, la solución óptima puede contener algunas posiciones vacías:

- No pueden disponerse cajas sobre una posición vacía o, dicho de otra manera, las posiciones vacías tienen una capacidad γ igual a 0, pero sí puede haber otras posiciones vacías sobre ella.
- La prioridad de una posición vacía se considera igual a $+\infty$, es decir, la mínima posible.
- El coste de una posición vacía es 0.

Por ejemplo, la solución óptima de la Figura 3 se ha obtenido con los datos de entrada:

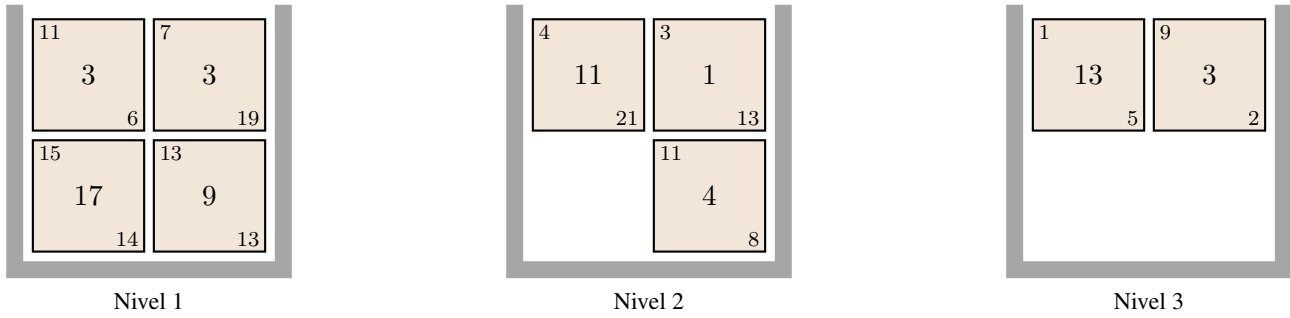


Figura 3: Disposición óptima de 9 cajas sobre un *pallet* de lado $l = 2$ con un máximo de tres niveles y un único lado de acceso. Cada caja muestra su prioridad en el centro, su peso en la esquina superior izquierda y su capacidad en la esquina inferior derecha. Las posiciones vacías se dejan en blanco. El nivel 1 es el inferior y el nivel 3 es el superior.

```

3 2
1 3 3 13 9 11 17 4 3
3 11 9 1 13 4 15 11 7
13 6 2 5 13 21 14 8 19

```

Se pide:

1. Modelar la tarea de optimización con un modelo general, con Programación Lineal entera, que minimice el coste medio de la disposición de todas las cajas, indicando claramente el tipo de problema de qué se trata, el significado de cada variable de decisión, el propósito de cada restricción, y la formalización de la función objetivo.

Está expresamente prohibida la escritura de restricciones, o la función objetivo con operadores que no sean puramente lineales.

2. Implementar un modelo general que sirva para resolver la tarea de optimización propuesta, que se debe llamar `parte-3.mod`.
3. Desarrollar un *script* en Python llamado `sol-3.py`, de acuerdo a las indicaciones dadas, que sirva para generar ficheros de datos `.dat`, y para resolver óptimamente la tarea de Programación Lineal resultante.

2.4. Parte 4: Optimización del montaje en una cantidad arbitraria de *pallets* incompletos

En la última parte se pide considerar la existencia de una disponibilidad ilimitada de *pallets* sobre los que se deben disponer cajas de acuerdo a las mismas restricciones del apartado anterior, con especial atención al hecho de que ningún *pallet* puede contener más de h niveles.

En este caso, el montaje de cajas sobre un nuevo *pallet* incurre en un coste P que es igual para todos, de modo que el coste total se define como la suma del coste de la disposición volumétrica de las cajas de cada *pallet* por separado, definido como la suma del coste de cada caja, más el coste P de haber preparado cada *pallet*.

Para la resolución de esta parte, se recibirá un fichero con el formato:

```

<longitud del lado del pallet,  $l$ > <número de niveles,  $h$ >, <coste del montaje de un pallet,  $P$ >
< $p_1$   $p_2$  ...  $p_n$ >
< $\omega_1$   $\omega_2$  ...  $\omega_n$ >
< $\gamma_1$   $\gamma_2$  ...  $\gamma_n$ >

```

donde la primera fila añade al final el coste de montar cada *pallet*.

Usando los datos de entrada:

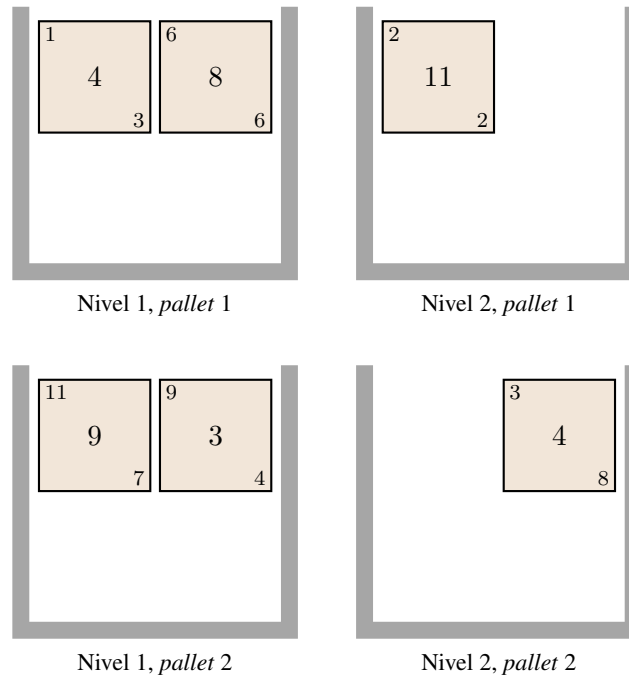


Figura 4: Disposición óptima de 6 cajas sobre dos *pallets* de lado $l = 2$, con un máximo de dos niveles y un único lado de acceso. Cada caja muestra su prioridad en el centro, su peso en la esquina superior izquierda y su capacidad en la esquina inferior derecha. Las posiciones vacías se dejan en blanco. El nivel 1 es el inferior. El coste de disposición de las cajas es cero y, con $P = 3$, el coste total es 6.

2	2	3
4	8	11
3	6	2
8	6	2

se obtiene la solución óptima mostrada en la Figura 4.

Se pide:

1. Modelar la tarea de optimización con un modelo general, con Programación Lineal entera, que minimice el coste total de la disposición de todas las cajas en la cantidad óptima de *pallets*, indicando claramente el tipo de problema de qué se trata, el significado de cada variable de decisión, el propósito de cada restricción, y la formalización de la función objetivo.

Está expresamente prohibida la escritura de restricciones, o la función objetivo con operadores que no sean puramente lineales.

2. Implementar un modelo general que sirva para resolver la tarea de optimización propuesta, que se debe llamar `parte-4.mod`.
3. Desarrollar un *script* en Python llamado `sol-4.py`, de acuerdo a las indicaciones dadas, que sirva para generar ficheros de datos `.dat`, y para resolver óptimamente la tarea de Programación Lineal resultante.

2.5. Parte 5: Análisis de Resultados

En este apartado se deben analizar todos los resultados obtenidos de cada parte, incluyendo análisis que demuestren una comprensión crítica de las soluciones mostradas.

Análisis de la complejidad de cada parte: ¿es posible resolver óptimamente alguna de las partes directamente sin usar Programación Lineal? ¿es posible que algunas instancias sean infactibles o no acotadas? ¿Es posible que haya más de una solución óptima?

Crea diferentes casos con el uso del *script*, variando los objetos de entrada y el valor de los parámetros. ¿Cuántas variables y restricciones obtienes en cada caso? Explica cómo estas modificaciones afectan a la dificultad de resolución del problema resultante. ¿Qué influye en el tiempo de resolución? ¿cuáles son los tamaños máximos que podéis resolver en cada caso?

3. Directrices para la Memoria

La memoria debe entregarse en formato .pdf y tener un máximo de 15 hojas en total, incluyendo la portada, contraportada e índice, si los hubiera. Debe tratarse de un documento técnico que contenga, al menos:

1. Breve introducción explicando los contenidos del documento.
2. Por cada parte (secciones 2.1–2.4):
 - Descripción del modelo, argumentando las decisiones tomadas, usando para ello una notación algebraica correcta, clara y concisa que debe ser lo más general posible, siguiendo para ello las convenciones de la Programación Lineal.
 - La descripción del modelo de cada parte debe hacerse por separado indicando claramente el significado de cada parámetro y variable de decisión, el propósito de cada restricción junto con los comentarios necesarios que justifiquen su uso, y la formalización de la función objetivo.

3. Análisis de los resultados, según se indica en la sección 2.5.

El análisis consiste en un análisis crítico de las soluciones y el uso de medidas que soporten las conclusiones mostradas, así como otras que pueden obtenerse del estudio de la solución propuesta.

La memoria **no debe incluir código fuente** en ningún caso.

4. Evaluación

La evaluación de la práctica se realizará sobre 10 puntos. La distribución de puntos es la siguiente:

1. Parte 1, sección 2.1 (1,5 puntos)
 - Modelización del problema (0,75 puntos)
 - Implementación del modelo (0,75 puntos)
2. Parte 2, sección 2.2 (1,5 puntos)
 - Modelización del problema (0,75 puntos)
 - Implementación del modelo (0,75 puntos)
3. Parte 3, sección 2.3 (2,5 puntos)
 - Modelización del problema (1,25 puntos)
 - Implementación del modelo (1,25 puntos)
4. Parte 4, sección 2.4 (2,5 puntos)
 - Modelización del problema (1,25 puntos)
 - Implementación del modelo (1,25 puntos)
5. Parte 5, sección 2.5 (2 puntos):
 - Parte 1, sección 2.1 (0,5 puntos)

- Parte 2, sección 2.2 (0,5 puntos)
- Parte 3, sección 2.3 (0,5 puntos)
- Parte 4, sección 2.4 (0,5 puntos)

En la evaluación de la *modelización* del problema, un modelo correcto supondrá la mitad de los puntos. Para obtenerse el resto de puntos, la modelización del problema deberá:

- Ser formalizada correctamente en la memoria, usando para ello una notación algebraica correcta, clara y concisa.
- Detallar claramente la utilidad de cada variable y restricción, así como las colecciones, si las hubiera, de parámetros, y conjuntos.
- Justificar detalladamente en la memoria todas las decisiones de diseño tomadas con especial atención a las restricciones y la función objetivo, que deben estar claramente identificadas en relación con el enunciado y deben indicar por qué hacen lo que se espera de ellas.

En la evaluación de la *implementación* del modelo, un modelo correcto supondrá la mitad de los puntos. Para obtenerse el resto de puntos, la implementación del problema deberá:

- Hacer uso (en la implementación) de las capacidades que ofrecen las herramientas para que hacer/actualizar el modelo sea lo más sencillo posible (por ejemplo, utilizar `sets` en MathProg).
- Mantener el código y la redacción del modelo correctamente organizado y comentado. Los nombres deben ser descriptivos. Deberán añadirse comentarios en los casos en que sea preciso mejorar la legibilidad.

Importante: los modelos implementados en GLPK deben ser correctos. Esto es, han de funcionar y obtener soluciones óptimas a cada problema propuesto. En ningún caso se obtendrá una calificación superior o igual a la mitad indicada más arriba por un modelo que no lo haga.

Así mismo, se debe tener en cuenta que **se considera que la práctica no funciona correctamente, si bien el modelo no es capaz de resolver un caso razonable, o si el script en Python no funciona correctamente**, por ejemplo, porque genera ficheros de datos erróneos, o en rutas diferentes de la indicada, o no escribe correctamente la solución óptima, etc.

IA generativa: el uso de herramientas de Inteligencia Artificial para realizar la práctica está expresamente prohibido, si bien es posible usarlas con el propósito de mejorar o razonar sobre modelos desarrollados previamente. En particular, la sospecha del uso fraudulento de herramientas de este tipo, o cualquier otra consideración que anime a creer que algún miembro de un equipo, o ambos, no han realizado la práctica íntegramente, es motivo suficiente **para hacer una evaluación oral paralela a la corrección de esta práctica.**

5. Entrega

Se tiene de plazo para entregar la práctica hasta el 23 de Octubre, Viernes, a las 23:55. **Este límite es fijo y no se extenderá de ningún modo.**

Ten en cuenta que los entregadores de cada grupo han sido configurados para poder sobrescribir versiones anteriores. Por lo tanto, **para evitar problemas con la entrega final a última hora, haz una entrega en cuanto tengas una parte acabada, y haz tantas actualizaciones como sea preciso.**

Sólo un miembro de cada pareja debe subir los siguientes ficheros:

- Un único fichero `.zip` a través del punto de entrega de 'Aula Global' llamado "*Primera práctica*".

El fichero debe nombrarse `p1-NIA1-NIA2.zip`, donde NIA1 y NIA2 son los últimos 6 dígitos del NIA (rellenando con 0s por la izquierda si fuera preciso) de cada miembro de la pareja.

Ejemplo: `p1-054000-671342.zip`.

- La memoria debe subirse separadamente a través del punto de entrega Turnitin de 'Aula Global' llamado "*Primera práctica (sólo pdf)*".

La memoria debe llamarse `NIA1-NIA2.pdf` —después de sustituir adecuadamente los NIAs de cada estudiante como en el caso anterior.

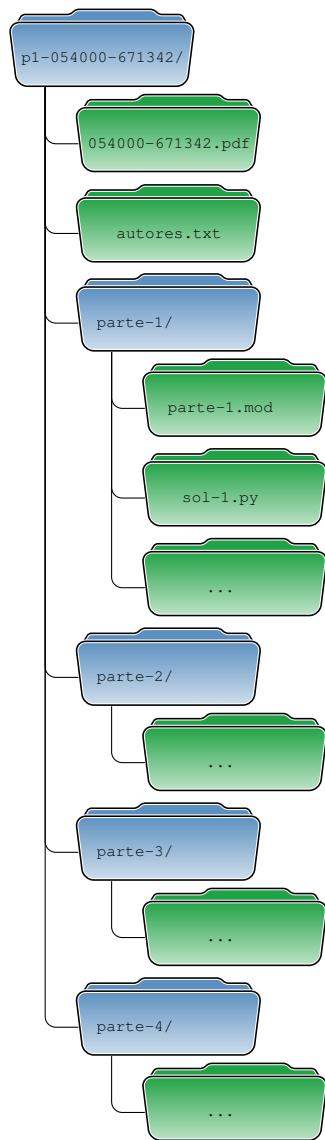
Ejemplo: `054000-671342.pdf`.

La descompresión del fichero `.zip` descrito en el primer punto debe producir un directorio llamado `p1-NIA1-NIA2`, donde NIA1 y NIA2 son los últimos 6 dígitos del NIA de cada estudiante relleno con 0s si fuera preciso. Este directorio debe contener: primero, la misma memoria entregada a través del segundo enlace descrito anteriormente que debe nombrarse como `NIA1-NIA2.pdf` —después de sustituir convenientemente los NIAs de cada estudiante; segundo, un fichero llamado `autores.txt`, codificado en UTF-8, que identifica a los miembros de cada pareja, con una línea por cada uno con el siguiente formato: NIA Apellidos, Nombre. Por ejemplo:

```
054000 Von Neumann, John
671342 Turing, Alan
```

Además, este directorio debe contener un directorio por cada parte realizada llamados "`parte-1`", "`parte-2`", "`parte-3`" y "`parte-4`". Las soluciones de cada parte (tanto el código fuente como cualquier otra información) deben incluirse separadamente en sus respectivos directorios.

La siguiente figura muestra una distribución posible de los ficheros después de descomprimir el fichero `.zip`:



en el que sólo se indican los contenidos de la primera parte —pero que sigue el mismo formato que las demás. Como se puede ver, cada directorio puede contener otros ficheros adicionales (indicados en la figura con puntos suspensivos). Son ejemplos válidos: ficheros adicionales de Python necesarios para la ejecución de los scripts; ficheros de datos de ejemplo de casos resueltos que se discutan en la memoria; código realizado en cualquier otro lenguaje de programación que implementa la resolución efectiva del caso, etc. pero en ningún caso deben entregarse ficheros generados por el solver de GLPK con la solución óptima.

Importante: no seguir cualquiera de las normas indicadas en el enunciado de esta práctica puede suponer una pérdida de hasta 2 puntos en la calificación final de la práctica.